

Python Cheat Sheet

For Reading & Writing Code — Beginner Friendly

THE MENTAL MODEL — How to Read Any Script

Thing	Think of it as	Example
Variable	A labeled box storing a value	<code>name = "Vickie"</code>
List	A grocery list of items	<code>fruits = ["apple", "mango"]</code>
Dictionary	A contact card (label + value)	<code>person = {"name": "Vickie"}</code>
Function	A reusable recipe	<code>def greet(): ...</code>
Loop	Do this for every item	<code>for item in list: ...</code>
If/Else	A decision branch	<code>if age > 18: ... else: ...</code>
Import	Grabbing a toolbox	<code>import requests</code>
Comment (#)	A human note, ignored by Python	<code># This does nothing</code>

■ *Golden Rule: Read top to bottom, like a recipe. Python does exactly what you wrote, in order.*

THE BASICS

Variables — labeled boxes

```
name = "Vickie"
age = 30
is_active = True # True or False is called a Boolean
```

Print — make the computer talk back

```
print("Hello!")
print(name) # prints: Vickie
print(f"Hi {name}, you are {age}!") # f-string: drops variables in
```

Lists — one variable, many items

```
fruits = ["apple", "banana", "mango"]
print(fruits[0]) # prints: apple (counting starts at 0!)
fruits.append("grape") # adds to the list
```

Dictionaries — label + value pairs

```
person = {
    "name": "Vickie",
    "city": "Atlanta"
}
print(person["name"]) # prints: Vickie
```

LOGIC & LOOPS

If / Else — making decisions

```
if age > 18:
    print("Adult") # indented = inside the if block
else:
    print("Not adult")
```

For Loop — do this for each item

```
for fruit in fruits:
    print(fruit) # prints each fruit one by one
```

While Loop — keep going until condition is false

```
count = 0
while count < 3:
    print(count) # prints 0, 1, 2
    count = count + 1
```

■■ Indentation matters! Python uses spaces/tabs to know what's inside an if, loop, or function. Always indent consistently.

FUNCTIONS — Reusable Recipes

Define once, use forever

```
def make_greeting(name):
    print("Hello, " + name + "!")

make_greeting("Vickie") # calls the function
make_greeting("Jordan") # use it again!
```

Functions that return a value

```
def add_numbers(a, b):
    return a + b

result = add_numbers(5, 3)
print(result) # prints: 8
```

IMPORTS — Grabbing Toolboxes

```
import requests # talk to APIs
import csv # read spreadsheet files
import os # interact with the operating system
import json # read/write JSON data
```

Library	What It Does	Common Use
requests	Talk to APIs / web services	Pull data from Okta, Jira, etc.
csv	Read/write CSV files	Open spreadsheet exports
os	Work with files & folders	List files, check paths
subprocess	Run terminal commands	Ping servers, run scripts
json	Read/write JSON data	API responses
smtplib	Send emails	Automated alerts
paramiko	SSH into servers remotely	Remote server management

READING ERRORS — Don't Panic!

Error Type	What It Means	Quick Fix
NameError	You used a variable that doesn't exist	Check your spelling
SyntaxError	Python can't read what you wrote	Look for missing : or)

IndentationError	Spacing is off	Check your tabs/spaces
TypeError	Wrong type of data used	Check if mixing text + numbers
KeyError	Dictionary key doesn't exist	Check your key spelling
IndexError	List position out of range	Remember: list starts at 0

■ Always read the *LAST* line of the error first — that's the actual problem.

THE IT OPS PATTERN — 80% of Scripts Do This

```
# 1. PULL data from somewhere
response = requests.get("https://api.service.com/users")
data = response.json()

# 2. DO something with it
for user in data["users"]:
    if user["active"] == False:
        print(f"Inactive user: {user['name']}")

# 3. SEND it somewhere / save it
with open("inactive_users.csv", "w") as file:
    file.write("name,email\n")
    file.write(f"{user['name']},{user['email']}\n")
```

Every IT automation script is basically: Pull → Do Something → Send/Save. Once you see that, you'll recognize it everywhere.